

計算機科学基礎実験 レポート

氏名：加藤悠菜 / 学籍番号：6325040

2026 年 5 月 19 日

1 各関数の動作説明

1. **checkl**: リスト x の中に要素 a が含まれているか判定する。見つければ `true`、なければ `false` を返す。
2. **dellt**: リスト x の先頭から n 個の要素を取り除いた残りのリストを返す。 n が 0 ならそのまま返し、負ならエラーとなる。
3. **dellt2**: リスト x の先頭から数えて a 番目の要素を削除したリストを返す。
4. **posl**: リストの n 番目の要素を返す。存在しない場合は例外を送出する。
5. **add2list**: 隣り合う 2 つの要素を足し合わせた新しいリストを返す。
6. **mullist**: 2 つのリストの同じ位置にある要素同士を掛け合わせたリストを返す。
7. **chglst**: リスト内の特定の要素を別の要素へ置換したリストを返す。
8. **replicate**: 指定された要素を n 個並べたリストを返す。
9. **inslist**: リストの n 番目に指定した要素を割り込ませたリストを返す。
10. **merge**: 2 つのリストを先頭から交互に組み合わせたリストを返す。
11. **inside_length**: リストの中に入っている各リストの要素数の合計を返す。
12. **concat**: リストのリストを連結して、1 つの平坦なリストにする。
13. **assoc**: タプルのリストから、指定したキーに対応する値を検索して返す。
14. **minimum**: リスト内の最小値を返す。
15. **extract**: 与えられた条件関数を満たす要素だけを抽出したリストを返す。
16. **index**: 指定した要素がリストの何番目 (0 から開始) にあるかを返す。
17. **numOfRotes**: 始点から終点までの最短経路 (右または上方向のみ) の総数を、再帰を用いて計算する。
18. **inter**: 2 つのリストを集合と見なし、共通する要素 (積集合) を返す。
19. **union**: 2 つのリストを集合と見なし、重複を許さない和集合を返す。
20. **diff**: 2 つのリストを集合と見なし、第 1 引数から第 2 引数の要素を除いた差集合を返す。

2 補助関数

課題 18,19,20 を実装するにあたり、共通して使用する以下の補助関数を定義した。

```
let rec mem a x=
match x with
[] -> false
|h :: t -> if h = a then true else mem a t
;;
```

これは要素 a がリスト x に含まれるか判定する関数である。

3 OCaml による定義と実行例

以下に作成したプログラムのソースコードおよび実行例を示す。

```
(* 1. check1 *)
let rec check1 a x=
match x with
  [] -> false
|h :: t ->
  if h = a then true
  else check1 a t
;;

(* 2.dellt *)
let rec dellt n x=
match x with
  [] -> []
|h :: t ->
  if n=0 then x
  else if n<0 then failwith "Error"
  else dellt (n-1) t
;;

(* 3.dellt2 *)
let rec dellt2 a x=
match x with
  [] -> []
|h :: t ->
  if a=1 then t
  else h :: dellt2 (a-1) t
;;
```

```

(* 4.posl *)
let rec posl n x=
match x with
[] -> failwith "Not Exist..."
|h :: t ->
  if n=1 then h
  else if n<1 then failwith "Not Exist..."
  else posl (n-1) t
;;

(* 5.add2list *)
let rec add2list x=
match x with
[] -> []
|h1 :: h2 :: t -> (h1+h2) :: add2list(h2 :: t)
|_ -> []
;;

(* 6.mullist *)
let rec mullist x y=
match (x,y) with
(h1 :: t1, h2 :: t2) -> (h1 * h2) :: mullist t1 t2
|_ -> []
;;

(* 7.chglist *)
let rec chglist (a,b) x=
match x with
[] -> []
|h :: t ->
  if h=a then b :: chglist (a,b) t
  else h :: chglist (a,b) t
;;

(* 8.replicate *)
let rec replicate n a=
if n=0 then []
else a :: replicate (n-1) a
;;

```

```

(* 9.inslist *)
let rec inslist n a x=
if n<1 then failwith "Error"
else match x with
[] -> [a]
|h :: t ->
  if n=1 then a :: x
  else h :: inslist (n-1) a t
;;

(* 10.merge *)
let rec merge x y=
match (x,y) with
([],[]) -> []
|([],y) -> y
|(x,[]) -> x
|(h1 :: t1,h2 :: t2)->
  h1 :: h2 :: merge t1 t2
;;

(* 11.inside_length *)
let rec inside_length x=
match x with
[] -> 0
|h ::t ->
  (List.length h)+inside_length t
;;

(* 12.concat *)
let rec concat x=
match x with
[] -> []
|h :: t ->
  h @ concat t
;;

```

```

(* 13.assoc *)
let rec assoc a x=
match x with
[] -> failwith "Not found..."
|(key,value) :: t ->
  if key=a then value
  else assoc a t
;;

```

```

(* 14.minimum *)
let rec minimum x=
match x with
[] -> failwith "Error"
|[h] -> h
|h :: t ->
  let m = minimum t in
  if h<m then h
  else m
;;

```

```

(* 15.extract *)
let rec extract f x=
match x with
[] -> []
|h :: t ->
  if f h then h :: extract f t
  else extract f t
;;

```

```

(* 16. index *)
let rec index x a =
  match x with
  | [] -> failwith "Not found"
  | h :: t ->
    if h = a then 0
    else 1 + index t a
;;

```

```

(* 17. numOfRotes *)
let rec numOfRotes (m, n) =
  if m = 0 || n = 0 then 1
  else numOfRotes (m - 1, n) + numOfRotes (m, n - 1)
;;

```

```

(* 18. inter *)
let rec inter x y =
match x with
| [] -> []
| h :: t ->
  if mem h y then h :: inter t y
  else inter t y
;;

```

```

(* 実行例 *)
# inter [1;2;3] [2;3;4];;
- : int list = [2; 3]

```

```

(* 19.union *)
let rec union x y=
match y with
[] -> x
|h :: t ->
  if mem h x then union x t
  else h :: union x t
;;

```

```

(* 実行例 *)
# union [1;2] [2;3];;
- : int list = [3; 1; 2]

```

```
(* 20.diff *)  
let rec diff x y=  
match x with  
[] -> []  
|h :: t ->  
  if mem h y then diff t y  
  else h :: diff t y  
;;
```

```
(* 実行例 *)  
# diff [1;2;3] [2;4];;  
- : int list = [1; 3]
```